

KaLM-Reranker-V1: Fast but Not Late Interaction for Compressed Document Reranking

Xinping Zhao^{1,2}, Jiaxin Xu¹, Ziqi Dai¹, Xin Zhang¹, Shouzheng Huang¹, Danyu Tang¹,
Xinshuo Hu, Meishan Zhang¹, Baotian Hu^{1,2*}, Min Zhang^{1,2}

¹Harbin Institute of Technology (Shenzhen); ²Shenzhen Loop Area Institute (SLAI)
zhaoxinping@stu.hit.edu.cn, mason.zms@gmail.com
hubaotian@hit.edu.cn, zhangmin2021@hit.edu.cn

Abstract

As retrieval systems scale, high-quality reranking becomes increasingly important. However, most existing rerankers, whether encoder-based or decoder-based, jointly encode the query and passage, tightly coupling their computation and limiting deployment efficiency as well as flexibility. We present KaLM-Reranker-V1, a *fast but not late-interaction* (FBNL) reranker that decouples query and passage computation while retaining expressive relevance modeling. Built on an encoder-decoder architecture, KaLM-Reranker-V1 uses the encoder to pre-encode passages with Matryoshka embedding pooling, while the decoder models the system instruction, user instruction, and query intent; cross-attention then captures relevance between the query context and passage representations. This design makes KaLM-Reranker-V1 efficient through decoupled passage encoding, yet not late interaction, by preserving rich relevance modeling through cross-attention. We instantiate KaLM-Reranker-V1 in three sizes, *Nano*, *Small*, and *Large*, with 0.27B, 1B, and 4B activated parameters, respectively. Extensive experiments on BEIR, MIRACL, and LMEB demonstrate that KaLM-Reranker-V1 achieves strong reranking performance with superior efficiency. On BEIR, KaLM-Reranker-V1 achieves state-of-the-art performance, on par with strong industrial models such as the Qwen3-Reranker series; on MIRACL, despite not being extensively trained on multilingual data, KaLM-Reranker-V1 still shows excellent reranking performance. Moreover, on LMEB, reranking models demonstrate a clear advantage, with even the 0.27B Nano model remaining competitive with 7–12B embedding models².

1 Introduction

Neural retrieval systems typically follow a two-stage pipeline: a retrieval stage first recalls a small set of candidates from a large corpus, and a reranking stage then performs finer-grained relevance modeling to produce the final ranking [Nogueira and Cho, 2019, Karpukhin et al., 2020]. The resulting ranked passages are then provided as input to large language models (LLMs) or further processed through context engineering. Reranking quality therefore plays a central role in end-to-end systems, especially in search [Nogueira and Cho, 2019], recommendation [Liu et al., 2022, Zhao et al., 2024a], and retrieval-augmented generation (RAG) [Lewis et al., 2020, Zhao et al., 2024b].

Most existing rerankers [Zhang et al., 2025c, Wang et al., 2025, Chen et al., 2024], however, tightly couple query and passage computation by modeling relevance from jointly processed query–passage inputs. As a result, passage representations typically have to be recomputed for each new query, substantially increasing online computation, making it hard to pre-compute passage representations offline and reducing efficiency at scale. One natural alternative is late interaction [Khattab and

*Corresponding Author

²Models are available at <https://huggingface.co/collections/KaLM-Embedding/lychee-kalm-reranker>.

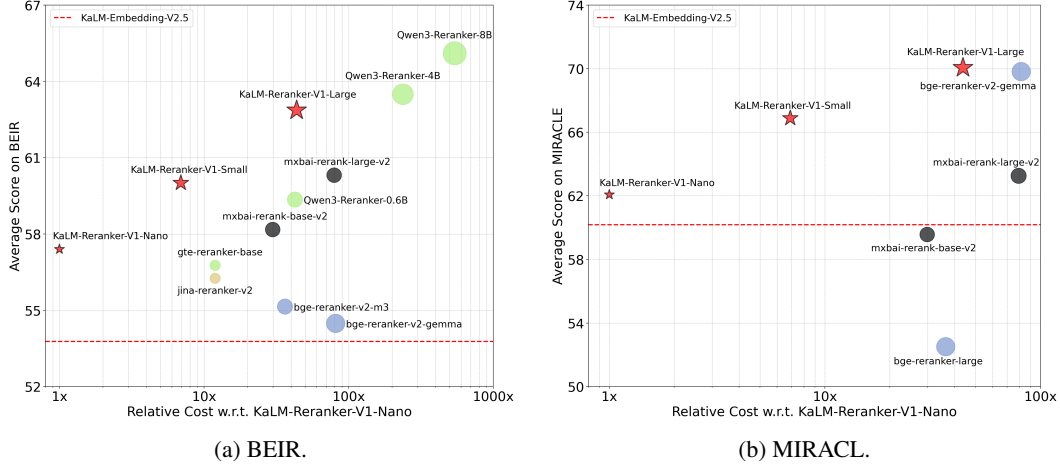


Figure 1: Comparison between the KaLM-Reranker-V1 series and other reranking models on BEIR and MIRACL in terms of reranking performance and relative online computation cost. The cost is estimated following the analysis in §5.1; the x-axis is plotted on a logarithmic scale. For a fair comparison on MIRACL, we exclude models that have been extensively trained on multilingual data. Marker sizes are proportional to model sizes, and points closer to the upper-left region indicate better performance–cost trade-offs. The results are taken from Tables 4 and 5.

Zaharia, 2020], which decouples query and passage encoding and improves efficiency at inference time. However, because scoring is based on similarity operations over independently encoded tokens, it limits fine-grained query–passage interaction.

In this work, we introduce the KaLM-Reranker-V1 series, which features a *fast but not late-interaction* (FBNL) reranking design that decouples query and passage computation while preserving expressive relevance modeling. KaLM-Reranker-V1 is built on an encoder–decoder architecture, where the encoder is dedicated to passage representation and supports offline document encoding. To further reduce storage and serving costs, it is trained with Matryoshka embedding pooling (MEP), allowing compact passage representations to remain effective for reranking. On the query side, the decoder takes the system instruction, user instruction, and query as inputs. Relevance is then computed through the decoder’s merged self-attention and cross-attention over the concatenation of the query context and pre-encoded passage representations, enabling query-conditioned interaction without fully re-encoding the passage for each request. Overall, this design has three key advantages:

- **Efficiency:** Passage representations are precomputed offline to reduce online serving costs.
- **Expressiveness:** The decoder supports richer relevance modeling than late interaction.
- **Compactness:** Matryoshka embedding pooling reduces storage while preserving reranking quality.

We instantiate KaLM-Reranker-V1 in three sizes: *Nano*, *Small*, and *Large*, with 0.27B, 1B, and 4B activated parameters, respectively, to meet diverse performance and serving-load requirements. Experimental results on BEIR [Thakur et al., 2021], a widely used English retrieval benchmark, MIRACL [Zhang et al., 2023], a multilingual retrieval benchmark spanning 18 languages, and LMEB [Zhao et al., 2026b], a long-horizon memory retrieval benchmark, show that KaLM-Reranker-V1 achieves comparable or better reranking performance than similarly sized models while substantially improving inference efficiency. As shown in Figure 1, KaLM-Reranker-V1 achieves higher performance at similar computational cost or substantially lower cost at comparable performance. For example, on BEIR (Figure 1a), KaLM-Reranker-V1-Nano slightly outperforms gte-reranker-base while improving efficiency by about 10x. On MIRACL (Figure 1b), despite not being extensively trained on multilingual data, KaLM-Reranker-V1-Large still outperforms bge-reranker-v2-gemma while improving efficiency by nearly 2x. With Matryoshka embedding pooling, performance decreases gradually as the compression ratio increases from 2x to 32x, while remaining within an acceptable range overall; at 2x and 4x compression, the performance drop is nearly negligible. In-depth analysis further shows that moderate compression largely preserves the model’s ability

to distinguish relevant from irrelevant passages, while overly large compression causes larger AUC drops, especially for smaller models. Last but not least, on LMEB, reranking models demonstrate clear advantages in memory retrieval, with even KaLM-Reranker-V1-Nano achieving performance competitive with 7–12B embedding models while using only 0.27B activated parameters.

2 Related Work

Embedding Models. Text embedding models map text into continuous vectors for efficient similarity search [Muennighoff et al., 2023, Enevoldsen et al., 2025, Zhang et al., 2025b]. From GloVe word vectors [Pennington et al., 2014] to transformer-based sentence embedders such as Sentence-BERT [Reimers and Gurevych, 2019], dense representations have been widely adopted for semantic similarity search. DPR [Karpukhin et al., 2020] further demonstrated that dual-encoder passage retrieval can serve as an effective and scalable first-stage retriever for open-domain QA. Subsequent models such as Contriever [Izacard et al., 2022], GTR [Ni et al., 2021], and E5 [Wang et al., 2022] improved generalization through contrastive pretraining, large-scale supervision, and stronger backbones. Recent LLM-driven embedding models have become dominant, including GTE [Li et al., 2023b, Zhang et al., 2024], Qwen3-Embedding [Zhang et al., 2025c], KaLM [Hu et al., 2025, Zhao et al., 2026a], BGE [Chen et al., 2024], Jina [Sturua et al., 2024, Akram et al., 2026], NV-Embed [Lee et al., 2025], and F2LLM [Zhang et al., 2026b]. Despite the strong performance of LLM-based embeddings, these models typically adopt a bi-encoder architecture, where queries and documents are encoded separately and matched only through vector similarity. This design limits fine-grained query–document interaction, making rerankers necessary to further refine the retrieved candidates.

Reranking Models. Reranking addresses the limited interaction in embedding retrieval by performing deeper query–document relevance modeling. Early neural rerankers such as BERT reranking concatenate the query and passage and apply full self-attention for relevance estimation [Nogueira and Cho, 2019]. Generative and LLM-based rerankers, including RankGPT [Qin et al., 2024], RankVicuna [Pradeep et al., 2023], BGE [Chen et al., 2024], Qwen3-Reranker [Zhang et al., 2025c], Jina-Reranker [Wang et al., 2025], and lychee-rerank [Zhang et al., 2026a], further strengthen this paradigm through generative scoring, instruction tuning, and stronger backbones. Despite their effectiveness, these methods tightly couple query and passage computation, requiring each candidate to be recomputed for every query. In parallel, late-interaction methods such as ColBERT [Khattab and Zaharia, 2020] and ColBERTv2 [Santhanam et al., 2022] decouple query and passage encoding while preserving token-level matching. They improve efficiency, but their interaction is largely limited to similarity aggregation between independently contextualized query and passage token representations. Overall, reranking methods have developed along multiple directions, including full interaction, generative scoring, and efficient late interaction. However, prevailing models often face a trade-off between efficiency and effectiveness [Zhang et al., 2025c, Wang et al., 2025]. KaLM-Reranker-V1 introduces a new reranking paradigm that alleviates this trade-off issue: the encoder produces reusable passage representations, while the decoder computes relevance through fine-grained interaction.

3 Model Architecture

3.1 Architecture

KaLM-Reranker-V1 is built on T5Gemma2 [Zhang et al., 2025a] foundation models and is instantiated in three encoder–decoder sizes: 270M–270M, 1B–1B, and 4B–4B. We initialize the models from T5Gemma2 to leverage its text modeling and instruction-following capabilities, while adapting the architecture for efficient document reranking. Table 1 details the model layers, activated parameters, document token dimension, and sequence length of each model configuration.

Figure 2 shows the architecture of KaLM-Reranker-V1 that addresses fundamental limitations in existing models. Specifically, current rerankers [Chen et al., 2024, Wang et al., 2025, Zhang et al., 2025c] typically couple query and document, requiring each query–document pair to be processed jointly and making large-scale serving expensive. Late-interaction methods such as ColBERT [Khattab and Zaharia, 2020] improve efficiency via separate query and document encoding followed by multi-vector interaction, but cannot capture deep query–document interactions during encoding.

Our *fast but not late-interaction* (FBNL) design departs from both encoder-/decoder-based rerankers and late-interaction models: unlike the former, it reuses passage representations for efficient serving;

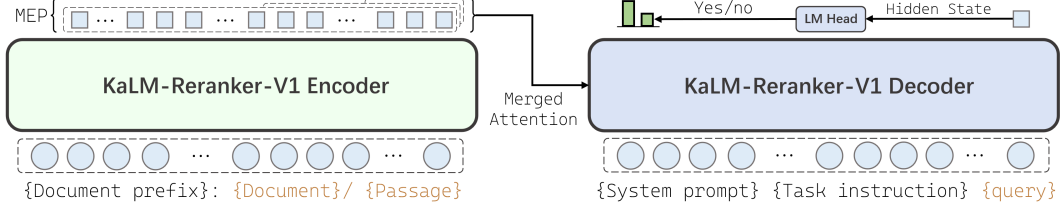


Figure 2: The overall framework of KaLM-Reranker-V1. The encoder produces compressed representations with MEP, and the decoder computes fine-grained relevance scores via merged attention.

Models	Activated Params.	Non-Embedding Params.	Embedding Params.	#Layers	Sequence Length	Document Token Dim.	MEP Support	Instruction Aware
KaLM-Reranker-V1-Nano	0.27B	100M	168M	18	128K	640	1x—32x	Yes
KaLM-Reranker-V1-Small	1B	698M	302M	26	128K	1152	1x—32x	Yes
KaLM-Reranker-V1-Large	4B	3209M	675M	34	128K	2560	1x—32x	Yes

Table 1: The KaLM-Reranker-V1 series. Activated parameters denote the number of parameters activated during inference, excluding the encoder parameters. Document token dimension is the encoded token size. MEP denotes Matryoshka embedding pooling, with supported compression ratios. “Instruction Aware” indicates whether task-specific instructions are supported.

unlike the latter, it preserves rich query–document interaction through decoder cross-attention instead of postponing interaction until after separate encoding. Together with Matryoshka token pooling in §4, these designs make KaLM-Reranker-V1 both efficient and expressive, enabling scalable reranking without sacrificing rich interaction.

Concretely, given an instruction I , a query q , and a candidate document p , the encoder maps p to a reusable representation:

$$\mathbf{H}_p = \text{Enc}(p) \in \mathbb{R}^{n \times d}, \quad (1)$$

where n is the document sequence length and d is the hidden dimension. The decoder takes the system instruction (omitted from the equation for brevity), the task instruction I , and the user query q as input, and attends to $\mathbf{H}_p$ through cross-attention to produce token logits over the vocabulary:

$$\mathbf{Z} = \text{Dec}(I, q; \mathbf{H}_p) \in \mathbb{R}^{|\mathcal{V}|}, \quad (2)$$

where $\mathcal{V}$ denotes the token vocabulary. More specifically, let $\mathbf{X} \in \mathbb{R}^{m \times d}$ denote the decoder input. The decoder forms queries from $\mathbf{X}$, and forms keys and values from the concatenation of the decoder input $\mathbf{X}$ and the encoder output $\mathbf{H}_p$:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = [\mathbf{X}; \mathbf{H}_p]\mathbf{W}_K, \quad \mathbf{V} = [\mathbf{X}; \mathbf{H}_p]\mathbf{W}_V, \quad (3)$$

where $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ are the query, key, and value projection matrices, respectively. The decoder representation is then computed as:

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}} + \mathbf{M}\right)\mathbf{V}\mathbf{W}_O, \quad (4)$$

where d_h denotes the attention head dimension, $\mathbf{M} \in \mathbb{R}^{m \times (m+n)}$ masks invalid attention positions, and $\mathbf{W}_O$ is the output projection matrix. The language-model head then projects the decoder representation at the first prediction position into the vocabulary logit vector $\mathbf{Z} \in \mathbb{R}^{|\mathcal{V}|}$.

To compute the relevance score, we compare the likelihood of generating yes versus no as the next token. At the first prediction position, we use only the two entries in $\mathbf{Z}$ corresponding to these label tokens, denoted as z_{yes} and z_{no} , respectively. The relevance score is defined as follows:

$$\text{score}(q, p) = \frac{\exp(z_{\text{yes}})}{\exp(z_{\text{yes}}) + \exp(z_{\text{no}})}. \quad (5)$$

Here, $\text{score}(q, p)$ denotes the relevance score of document p with respect to query q .

3.2 Prompt Template

Table 2 summarizes the prompt templates used by KaLM-Reranker-V1. The encoder receives only the **document** during reranking training, producing a reusable passage representation that can be

Prompt Templates for KaLM-Reranker-V1

Prompt Template for Encoder

<Document>: {Document}

Prompt Template for Decoder

<bos><start_of_turn>user

Judge whether the Document meets the requirements based on the Query and the Instruct provided. Note that the answer can only be “yes” or “no”.

<Instruct>: {Instruction}

<Query>: {Query}<end_of_turn>

<start_of_turn>model\n\n\n\n

Table 2: Prompt templates for KaLM-Reranker-V1. The encoder takes the document for reusable passage encoding; the decoder takes the instruction and query to produce the reranking score.

stored and shared across queries. Because T5Gemma2 does not use a system role, the system prompt (*i.e.*, “Judge whether ...”) is placed directly on the **user** side, together with the **instruction** and **query**; the **model** side then predicts whether the encoded document satisfies the specified requirements. This separation aligns the template with the FBNL design: document representations remain reusable, while the decoder performs instruction- and query-aware interaction through cross-attention at scoring time. The decoder output is constrained to the binary label tokens “yes” and “no”, which are used to compute the reranking score.

4 Model Training

Details of the training data, including data sources and preprocessing, are provided in Appendix A.

4.1 Training Objective

Supervised Fine-Tuning (SFT). Given an instruction I , a query q , and a candidate document p , KaLM-Reranker-V1 predicts a binary relevance label $l \in \{\text{yes}, \text{no}\}$. Following the scoring method in §3, we compute the logits of the two label tokens and optimize the supervised fine-tuning objective:

$$\mathcal{L}_{\text{sft}}(I, q, p, l) = -\log \frac{\exp(z_l)}{\exp(z_{\text{yes}}) + \exp(z_{\text{no}})}, \quad (6)$$

where z_l is the logit of the ground-truth label token. The label is yes for relevant documents and no for irrelevant documents. Although reusable passage representations enable efficient serving, storing the full representation $\mathbf{H}_p \in \mathbb{R}^{n \times d}$ for every passage in a large corpus can impose substantial storage overhead. To reduce this cost, we introduce **Matryoshka Embedding Pooling (MEP)**, which compresses passage representations along the sequence dimension while preserving reranking effectiveness. For a compression ratio r , MEP groups every r consecutive passage tokens and applies a simple yet effective mean pooling (*i.e.*, $\text{MeanPool}(\cdot)$) to obtain one compact token representation:

$$\mathbf{H}_p^{(r)}[j] = \text{MeanPool}(\mathbf{H}_p[(j-1)r+1 : \min(jr, n)]), \quad j = 1, \dots, \left\lceil \frac{n}{r} \right\rceil, \quad (7)$$

where $\mathbf{H}_p^{(r)} \in \mathbb{R}^{\lceil n/r \rceil \times d}$ denotes the compressed passage representation, and $\lceil \cdot \rceil$ denotes the ceiling function. Having established $\mathbf{H}_p^{(r)}$, the decoder computes the relevance score at compression ratio r :

$$\text{score}^{(r)}(q, p) = \frac{\exp(z_{\text{yes}}^{(r)})}{\exp(z_{\text{yes}}^{(r)}) + \exp(z_{\text{no}}^{(r)})}. \quad (8)$$

During reranking training, we optimize this objective over a set of compression ratios, *e.g.*, $\mathcal{R} = \{1, 2, 4, 8, 16, 32\}$, where $r = 1$ corresponds to the vanilla representation. The final objective is:

$$\mathcal{L}_{\text{sft}}(I, q, p, l, \mathcal{R}) = \sum_{r \in \mathcal{R}} \lambda_r \left(-\log \frac{\exp(z_l^{(r)})}{\exp(z_{\text{yes}}^{(r)}) + \exp(z_{\text{no}}^{(r)})} \right), \quad (9)$$

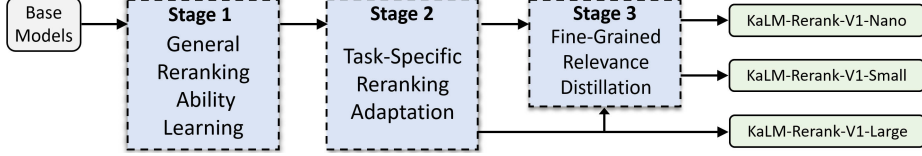


Figure 3: Progressive multi-stage training pipeline of the KaLM-Reranker-V1 series.

where λ_r controls the loss weight for compression ratio r and is set to 1 by default. This objective encourages the model to learn passage representations that remain effective across compression ratios, enabling flexible trade-offs between memory cost and reranking quality.

Knowledge Distillation (KD). For distillation training [Hinton et al., 2015, Rao et al., 2024], we use a stronger teacher reranker to provide soft supervision. Concretely, for each query–document pair (q, p) , the teacher produces a soft label $y \in [0, 1]$, while the student predicts a score $\hat{y} = \text{score}(q, p)$. Given the teacher soft labels and student scores, we optimize binary cross-entropy with soft labels:

$$\mathcal{L}_{\text{kd}} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y}), \quad (10)$$

where y denotes the teacher’s soft label and $\hat{y}$ denotes the student score. Compared with the hard binary labels used in Equations 6 and 9, soft labels retain fine-grained relevance signals and largely mitigate the effect of potential false hard negatives during reranking training.

4.2 Multi-stage Training

Multi-stage training has been widely adopted to improve the model generalization as well as task adaptation of embedding models [Zhang et al., 2025c, Akram et al., 2026, Zhao et al., 2026a, Zhang et al., 2026b]. Motivated by this practice, we train the KaLM-Reranker-V1 series using a progressive three-stage training pipeline, as illustrated in Figure 3. The pipeline gradually progresses from (1) instruction-free general reranking learning to (2) task-specific adaptation and (3) nuanced relevance discrimination, as described below.

- **General Reranking Ability Learning.** In the first stage, we train the model without explicit task instructions. This stage aims to establish a domain-agnostic reranking foundation, enabling the model to learn general reranking ability across diverse retrieval scenarios. By relying only on query–document relevance signals, the model learns to produce robust binary relevance judgments, which improves its generalization before task-specific adaptation.
- **Task-Specific Reranking Adaptation.** In the second stage, we introduce task-specific instructions and continue supervised fine-tuning on higher-quality reranking data. These task instructions specify the search intent and relevance criterion, allowing the model to adapt its scoring behavior to each retrieval setting³. This stage improves task-aware reranking performance and better aligns the model with real-world application scenarios.
- **Fine-Grained Relevance Distillation.** In the final stage, we further refine the model with fine-grained soft label signals produced by a stronger teacher reranker⁴. Different from hard yes/no labels, soft labels provide graded relevance signals and preserve nuanced relevance differences among candidate documents. This stage improves KaLM-Reranker-V1’s discriminative ability, especially for hard negatives and borderline relevance cases.

5 Complexity Analysis

5.1 Time Complexity

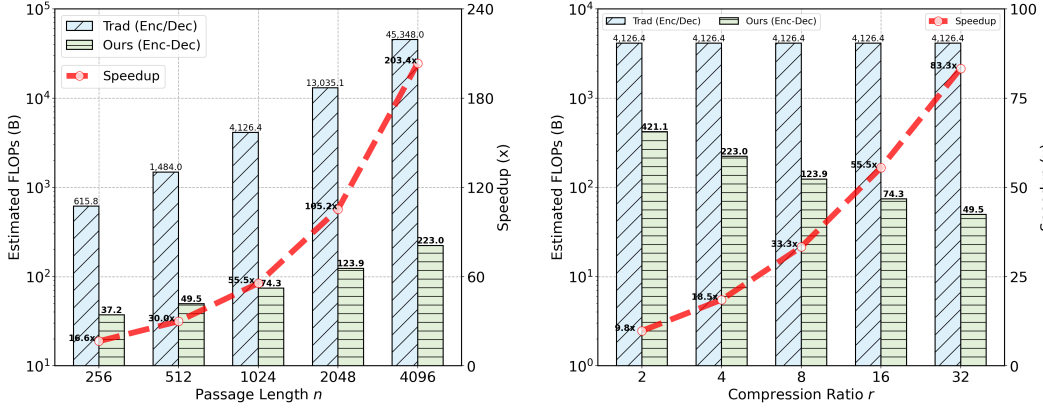
We compare the online time complexity of KaLM-Reranker-V1 with conventional encoder-based and decoder-based rerankers as shown in Table 3. Let K denote the number of candidate passages to rerank, $|q|$ the query length, n the average passage length, and d the hidden dimension. Let L denote the number of Transformer layers in a conventional reranker, and r the MEP compression ratio.

³For example, **ArguAna** [Wachsmuth et al., 2018, Thakur et al., 2021] uses the task instruction: “Given a claim, find documents that refute the claim.”

⁴Specifically, we use KaLM-Reranker-V1-Large as the teacher model for training KaLM-Reranker-V1-Small&Nano.

Model Type	Interaction Pattern	Time Complexity
Encoder-based reranker	Bidirectional attention	$\mathcal{O}(KL((q + n)^2d + (q + n)d^2))$
Decoder-based reranker	Causal attention	$\mathcal{O}(KL((q + n)^2d + (q + n)d^2))$
Enc-Dec-based reranker (ours)	Merged self-attn. + cross-attn.	$\mathcal{O}(\frac{L}{2}K(q (q + \lceil \frac{n}{r} \rceil)d + (q + \lceil \frac{n}{r} \rceil)d^2))$

Table 3: Serving time complexity comparison for reranking K candidate passages. Conventional encoder-based and decoder-based rerankers process each query–passage pair online, whereas our encoder–decoder reranker reuses cached passage representations pre-encoded by the encoder and performs merged attention in the decoder over the query context and compressed passage tokens.



(a) Varying passage length n with $|q| = 32$, $r = 16$, $K = 100$, $d = 640$, and $L = 36$. (b) Varying compression ratio r with $|q| = 32$, $n = 1024$, $K = 100$, $d = 640$, and $L = 36$.

Figure 4: Serving time complexity comparison under different reranking settings, focusing on n and r , the two hyperparameters most frequently adjusted in deployment. Bars show the estimated FLOPs, and the line reports Speedup (x), the relative efficiency gain of our reranking paradigm over traditional rerankers. Here, “B” denotes billion, “Trad” denotes traditional, “Enc/Dec” denotes encoder-based/decoder-based rerankers, and “Enc-Dec” represents our encoder–decoder reranker.

For a Transformer layer with sequence length n , self-attention costs $\mathcal{O}(n^2d)$ and feed-forward/projection modules cost $\mathcal{O}(nd^2)$. We assume the encoder-based and decoder-based rerankers have the same total model size as KaLM-Reranker-V1. Since our model splits its parameters evenly between the encoder and decoder, its online decoder has approximately $L/2$ layers. The passage encoder of our model is used only for offline pre-encoding and is not invoked during online reranking.

Conventional rerankers jointly encode each query–passage pair online for relevance scoring. For each candidate, the joint input sequence length is $|q| + n^5$; therefore, reranking K candidates requires:

$$\mathcal{O}(KL((|q| + n)^2d + (|q| + n)d^2)). \quad (11)$$

In contrast, KaLM-Reranker-V1 runs only the decoder online and reuses fixed pre-encoded passage representations. In the merged self-attn. + cross-attn., query tokens attend to both query tokens and cached passage tokens. With MEP, each passage contributes only $\lceil n/r \rceil$ cached tokens, which remain unchanged during scoring. Thus, the online serving complexity of KaLM-Reranker-V1 requires:

$$\mathcal{O}\left(\frac{L}{2}K\left(|q|(|q| + \lceil \frac{n}{r} \rceil)d + \left(|q| + \lceil \frac{n}{r} \rceil\right)d^2\right)\right), \quad (12)$$

where **the first term** accounts for merged self-attn. + cross-attn. between query tokens and the combined query–passage tokens, and **the second term** accounts for linear projections and feed-forward layers for query and passage tokens. The offline passage encoding cost is $\mathcal{O}(K\frac{L}{2}(n^2d + nd^2))$ for K passages, but it is computed once and reused across queries, so it does not affect online serving.

Efficiency Gains. Figure 4 further quantifies the computational efficiency gains during deployment under representative settings. In Figure 4a, we fix $|q|$, r , K , d , and L to common values and vary

⁵We ignore prompt-template and instruction tokens, as their length is negligible compared with $|q| + n$.

the passage length n within the range of $\{256, 512, 1024, 2048, 4096\}$. Even for short passages with $n = 256$, our method achieves a $16.6\times$ efficiency gain. On the other hand, for long passages, the advantage becomes more significant: when $n = 4096$, our method achieves a $203.4\times$ efficiency gain. In Figure 4b, we fix $|q|$, n , K , d , and L to common values and vary the compression ratio r within the range of $\{2, 4, 8, 16, 32\}$. Even with a low compression ratio of $r = 2$, our method achieves nearly a $10\times$ efficiency gain; under commonly used compression ratios $r = 4$ and $r = 8$, the gains further increase to $18.5\times$ and $33.3\times$, respectively. In conclusion, these results show that offline passage encoding and MEP substantially reduce online overhead, yielding particularly large benefits for long passages and higher compression ratios.

5.2 Space Complexity

We analyze the additional space cost introduced by caching passages in KaLM-Reranker-V1 during large-scale reranking. Conventional encoder-based or decoder-based rerankers compute relevance by processing each query–passage pair online. As a result, they do not maintain reusable passage representations for the whole corpus, and their serving-time storage is mainly limited to model parameters and retrieved raw texts.

In contrast, our model caches pre-encoded passage representations to avoid repeated passage computation across queries, at the cost of an additional memory buffer. Suppose there are N passages, each with an average length of n tokens and hidden dimension d . If full passage representations are cached, the additional space cost is:

$$\mathcal{O}(Nnd), \quad (13)$$

where each passage stores $\mathbf{H}_p \in \mathbb{R}^{n \times d}$. For reranking a candidate list of size K , the online buffer for passage representations is $\mathcal{O}(Knd)$ before compression. This is the main space overhead of our model compared with encoder-based and decoder-based rerankers.

To control this overhead, KaLM-Reranker-V1 introduces **Matryoshka Embedding Pooling** (MEP) to compress passage representations along the sequence dimension. With compression ratio r , the cached representation of each passage becomes $\mathbf{H}_p^{(r)} \in \mathbb{R}^{\lceil n/r \rceil \times d}$, reducing the total passage cache to

$$\mathcal{O}\left(N \left\lceil \frac{n}{r} \right\rceil d\right), \quad (14)$$

and the online candidate buffer to $\mathcal{O}(K \lceil n/r \rceil d)$. Thus, MEP balances memory cost and reranking quality: larger r values reduce cached passage representations nearly linearly, while preserving decoder-side fine-grained relevance modeling.

6 Evaluation

The training and implementation details of KaLM-Reranker-V1 are provided in Appendix B.

6.1 Experimental Setup

Benchmarks. Our evaluation focuses on retrieval benchmarks that test different aspects of reranking capability. We use BEIR [Thakur et al., 2021] to assess out-of-domain generalization across 13 heterogeneous English retrieval tasks, ranging from question answering on HotpotQA [Yang et al., 2018] and duplicate-question retrieval on CQADupStack [Hoogeveen et al., 2015] to entity retrieval on DBPedia [Hasibi et al., 2017] and fact verification on FEVER [Thorne et al., 2018]. We exclude Touche2020 and TRECCOVID because their test sets contain only 49 and 50 queries, respectively, which makes the evaluation results less stable and robust. We use MIRACL [Zhang et al., 2023] to evaluate multilingual reranking robustness across 18 languages spanning diverse linguistic families, from Arabic and Bengali to Hindi and Swahili, which requires strong multilingual understanding. We further evaluate on the six dialogue memory retrieval tasks from LMEB [Zhao et al., 2026b], which assess retrieval and reranking ability in long-horizon dialogue memory scenarios. We adopt a two-stage retrieval and reranking pipeline for all experiments. In the first stage, KaLM-Embedding-V2.5 [Zhao et al., 2026a] is used as the dense retriever to retrieve the top-100 candidate passages for each query. In the second stage, all reranking models rescore the same top-100 candidates for fair comparison. We use nDCG@10 as the main evaluation metric. Evaluation instruction templates are available in Appendix C.

Models	Size	Cost	Avg.	AA	CF	CQA	DB	FV	FQA	HQA	MS	NF	NQ	QR	SD	SF
First-stage Retriever																
KaLM-Embedding-V2.5	0.5B	–	53.78	59.60	32.53	45.54	42.62	83.72	46.99	70.74	40.15	36.95	57.53	88.88	19.88	73.98
Second-stage Reranker																
<i>Models with more than 4B parameters</i>																
Qwen3-Reranker-8B	8B	539.7x	65.11	82.92	49.63	52.49	56.07	94.96	59.72	84.24	46.84	41.71	77.13	89.88	28.97	81.85
<i>Models with 1B–4B parameters</i>																
Qwen3-Reranker-4B	4B	236.8x	63.50	79.85	49.92	<u>49.94</u>	54.59	94.46	<u>56.69</u>	<u>83.68</u>	44.47	<u>41.57</u>	<u>74.03</u>	88.40	27.14	80.73
KaLM-Reranker-V1-L	4B	43.7x	<u>62.87</u>	<u>74.39</u>	38.56	53.05	<u>50.87</u>	92.86	62.63	84.10	<u>46.13</u>	41.99	75.72	91.00	<u>24.88</u>	81.12
bge-reranker-v2-gemma	2.5B	81.3x	54.49	69.34	29.60	37.28	43.02	82.87	47.21	82.09	45.38	28.59	68.80	86.64	18.86	68.66
mxbai-rerank-large-v2	1.5B	79.2x	60.32	71.43	<u>46.54</u>	45.90	49.74	<u>93.74</u>	50.75	82.66	47.65	37.59	70.77	<u>89.56</u>	16.99	<u>80.86</u>
<i>Models with 0.5B–1B parameters</i>																
KaLM-Reranker-V1-S	1B	6.9x	60.01	<u>67.48</u>	41.01	48.36	46.64	92.46	55.32	<u>82.64</u>	43.79	40.69	71.41	90.48	<u>21.88</u>	78.02
bge-reranker-large	0.6B	36.3x	51.86	28.47	41.51	38.89	48.76	<u>93.01</u>	41.41	83.12	<u>46.96</u>	34.14	52.46	73.84	17.23	74.41
bge-reranker-v2-m3	0.6B	36.3x	55.15	41.73	37.41	39.09	47.77	90.15	45.07	82.62	47.79	34.54	<u>69.25</u>	<u>89.14</u>	18.04	74.35
Qwen3-Reranker-0.6B	0.6B	42.4x	<u>59.36</u>	73.13	48.84	<u>47.21</u>	49.43	93.45	<u>47.99</u>	81.19	42.35	<u>39.32</u>	64.04	84.12	22.15	78.46
mxbai-rerank-base-v2	0.5B	29.8x	58.18	63.41	<u>43.38</u>	45.79	<u>48.97</u>	92.71	46.43	81.45	46.79	38.30	66.83	88.14	14.93	79.16
<i>Models with fewer than 0.5B parameters</i>																
gte-reranker-base	0.3B	11.9x	<u>56.77</u>	<u>58.44</u>	47.04	39.21	<u>47.31</u>	94.25	45.35	81.48	<u>45.44</u>	36.61	65.58	81.83	17.74	77.71
jina-reranker-v2	0.3B	11.9x	56.26	51.59	34.40	<u>40.85</u>	49.03	<u>92.35</u>	<u>46.10</u>	79.73	47.71	<u>37.12</u>	67.16	<u>87.80</u>	<u>20.01</u>	<u>77.50</u>
KaLM-Reranker-V1-N	0.27B	1.0x	57.41	64.22	<u>36.20</u>	45.62	46.10	91.34	48.14	<u>81.29</u>	43.30	37.56	<u>65.70</u>	89.52	20.55	76.84

Table 4: BEIR [Thakur et al., 2021] reranking results measured by nDCG@10. The 13 task abbreviations follow Table 9. “N”, “S”, and “L” denote Nano, Small, and Large, respectively. Avg. denotes the average performance across the 13 tasks. Cost denotes the estimated relative online computation cost derived from the time complexity analysis in §5.1, using $|q| = 32$, $n = 1024$, $K = 1$, and $r = 4$, with L and d obtained from Table 1 and Table 7, and normalized to KaLM-Reranker-V1-Nano as 1.0x. Within each parameter group, the best results are **boldfaced**, and the second-best results are underlined.

Baselines. We compare KaLM-Reranker-V1 with a spectrum of representative open-source rerankers with parameter sizes ranging from 0.3B to 8B, including bge-reranker-large/bge-reranker-v2-m3/bge-reranker-v2-gemma [Li et al., 2023a, Xiao et al., 2024, Chen et al., 2024], gte-multilingual-reranker-base (gte-reranker-base)/Qwen3-Reranker-0.6B/Qwen3-Reranker-4B/Qwen3-Reranker-8B [Zhang et al., 2025c, 2024], jina-reranker-v2-base-multilingual (jina-reranker-v2) [Jina AI, 2024], and mxbai-rerank-base-v2/mxbai-rerank-large-v2 [Li et al., 2025]. These baselines include widely used encoder- and decoder-based rerankers, as well as recent multilingual and LLM-based models, enabling comprehensive comparisons across architectures and scales. It is worth noting that, because KaLM-Reranker-V1 is currently trained mainly on Chinese and English corpora, for MIRACL we exclude models that have been extensively trained on multilingual data to ensure a fair comparison. Table 7 summarizes the public checkpoints as well as key model specifications of the embedding and reranking models used in our evaluation. Unless otherwise specified, KaLM-Reranker-V1 uses the default compression ratio of $r = 4$ in all experiments. Detailed evaluation hyperparameter settings are presented in Table 6.

6.2 Main Results

Tables 4 and 5 present the reranking performance of KaLM-Reranker-V1 and competitive baselines on BEIR and MIRACL. From the results, we mainly have the following observations:

KaLM-Reranker-V1 achieves strong reranking performance with superior efficiency. The KaLM-Reranker-V1 series performs on par with strong industrial rerankers from the *Qwen*, *BGE*, and *Jina* families. On BEIR, KaLM-Reranker-V1-Nano, KaLM-Reranker-V1-Small, and KaLM-Reranker-V1-Large achieve the best or second-best results on 9/13, 8/13, and 11/13 tasks, respectively. Beyond effectiveness, KaLM-Reranker-V1 is also highly efficient. For example, on BEIR, KaLM-Reranker-V1-Small outperforms Qwen3-Reranker-0.6B with a much lower online computation cost (6.9x vs. 42.4x).

KaLM-Reranker-V1 shows promising multilingual reranking ability despite limited multilingual training. Although KaLM-Reranker-V1 is not extensively trained on multilingual data, it still performs competitively on MIRACL. For example, KaLM-Reranker-V1-Large outperforms

Models	Size	Cost	Avg.	ar	bn	de	en	es	fa	fi	fr	hi
First-stage Retriever												
KaLM-Embedding-V2.5	0.5B	–	60.19	69.32	63.65	53.05	53.40	51.32	49.55	70.98	52.14	44.90
Second-stage Reranker												
<i>Models with more than 1B parameters</i>												
KaLM-Reranker-V1-Large	4B	43.7x	70.07	77.69	79.98	63.60	63.54	60.51	60.62	80.00	58.49	64.01
bge-reranker-v2-gemma	2.5B	81.3x	<u>69.82</u>	<u>75.43</u>	80.28	<u>62.27</u>	<u>63.53</u>	<u>60.42</u>	<u>60.30</u>	<u>77.45</u>	61.91	<u>62.32</u>
mxbai-rerank-large-v2	1.5B	79.2x	63.26	74.50	65.53	50.08	61.42	55.82	52.33	72.42	56.15	45.44
<i>Models with up to 1B parameters</i>												
KaLM-Reranker-V1-Small	1B	6.9x	66.89	<u>73.49</u>	77.06	58.10	62.09	58.83	56.43	77.81	57.21	59.54
mxbai-rerank-base-v2	0.5B	29.8x	59.58	73.65	57.16	29.12	60.86	<u>56.97</u>	47.74	69.56	<u>55.81</u>	42.68
bge-reranker-large	0.6B	36.3x	52.52	55.24	62.45	44.94	50.45	42.47	42.96	68.89	46.47	38.00
KaLM-Reranker-V1-Nano	0.27B	1.0x	<u>62.08</u>	69.00	<u>70.91</u>	<u>50.45</u>	60.41	56.93	<u>52.53</u>	<u>73.11</u>	51.93	<u>53.87</u>
Models	Size	Cost	Avg.	id	ja	ko	ru	sw	te	th	yo	zh
KaLM-Embedding-V2.5	0.5B	–	60.19	49.89	61.37	58.03	57.87	66.54	74.53	70.19	79.20	57.48
<i>Models with more than 1B parameters</i>												
KaLM-Reranker-V1-Large	4B	43.7x	70.07	<u>54.76</u>	<u>72.04</u>	<u>69.86</u>	73.57	78.25	<u>82.55</u>	81.57	88.66	51.54
bge-reranker-v2-gemma	2.5B	81.3x	<u>69.82</u>	57.26	72.34	70.42	<u>71.51</u>	<u>74.70</u>	83.10	<u>79.37</u>	<u>80.85</u>	63.36
mxbai-rerank-large-v2	1.5B	79.2x	63.26	53.50	71.92	66.96	69.97	63.75	57.90	79.35	79.06	<u>62.50</u>
<i>Models with up to 1B parameters</i>												
KaLM-Reranker-V1-Small	1B	6.9x	66.89	53.08	68.84	65.46	69.93	74.63	80.00	76.94	83.87	50.73
mxbai-rerank-base-v2	0.5B	29.8x	59.58	54.26	71.21	64.36	<u>66.54</u>	60.51	49.32	76.98	75.34	60.43
bge-reranker-large	0.6B	36.3x	52.52	37.96	42.19	46.41	44.29	57.98	70.03	64.18	75.16	<u>55.26</u>
KaLM-Reranker-V1-Nano	0.27B	1.0x	<u>62.08</u>	51.58	62.81	<u>65.02</u>	61.30	<u>64.63</u>	<u>73.67</u>	70.16	<u>78.22</u>	50.90

Table 5: MIRACL [Zhang et al., 2023] reranking results measured by nDCG@10. The first and second subtables report the first and second halves of the 18 MIRACL languages, respectively; language abbreviations follow Table 10. Avg. denotes the average performance across the 18 languages. Cost denotes the estimated relative online computation cost derived from the time complexity analysis in §5.1, using $|q| = 32$, $n = 1024$, $K = 1$, and $r = 4$, with L and d obtained from Table 1 and Table 7, and normalized to KaLM-Reranker-V1-Nano as 1.0x. Within each parameter group, the best results are **boldfaced**, and the second-best results are underlined.

bge-reranker-v2-gemma while being more efficient. On the Chinese subset of MIRACL, however, KaLM-Reranker-V1 remains relatively weak despite being trained with a substantial amount of Chinese data. This suggests that the foundation model’s Chinese capability may still be a bottleneck, and we will continue to improve it in the future.

The FBNL reranking paradigm offers both efficiency and expressive relevance modeling. Built on the FBNL paradigm and trained with 3.7M data (Table 14), the KaLM-Reranker-V1 series achieves performance comparable to the Qwen3-Reranker series on BEIR, which follows the vanilla reranking paradigm and is trained on 19M high-quality data [Zhang et al., 2025c]. Meanwhile, KaLM-Reranker-V1 achieves substantially higher efficiency, showing that FBNL preserves expressive relevance modeling while substantially reducing serving cost.

Evaluation results and analysis on LMEB [Zhao et al., 2026b] are presented in Appendix E.

6.3 Effectiveness of Matryoshka Reranking

Figure 5 and Tables 15 and 16 show the effect of Matryoshka embedding pooling with different compression ratios on BEIR and MIRACL. We analyze how reranking performance changes as passage representations become increasingly compressed and draw the following observations:

Reranking performance decreases as the compression ratio increases. Overall, performance decreases as the compression ratio increases because aggressive pooling discards some useful information. However, the degradation is relatively mild from $r = 2$ to $r = 16$, indicating that KaLM-Reranker-V1 can preserve most of its reranking effectiveness under moderate compression.

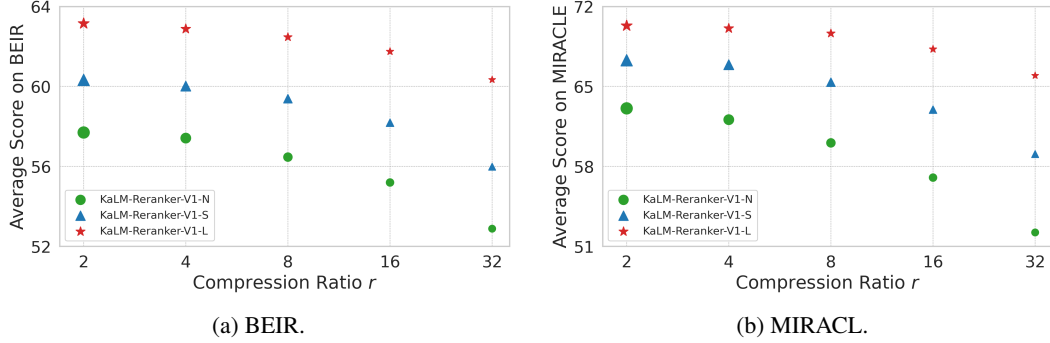


Figure 5: Reranking performance of KaLM-Reranker-V1 under different Matryoshka embedding pooling compression ratios on BEIR and MIRACL. Each point corresponds to a specific model size and compression ratio r . Results on individual tasks are presented in Tables 15 and 16.

The performance drop becomes much more substantial when increasing the compression ratio from $r = 16$ to $r = 32$. This is mainly because, due to memory constraints, our training only uses compression ratios $R = \{2, 4, 8, 16\}$. In conclusion, we recommend using $r = 2$ to $r = 8$ in practical deployment, which provides a favorable trade-off between effectiveness and efficiency.

Larger models are more robust to compression. As r increases, passage representations inevitably lose information, leading to lower reranking performance. In comparison, larger models show smaller performance drops under the same compression ratio. For example, KaLM-Reranker-V1-Large degrades much less than KaLM-Reranker-V1-Small and KaLM-Reranker-V1-Nano, suggesting that stronger model capacity helps preserve useful relevance modeling signals in compressed passage representations. This indicates that model size can partly compensate for representation compression, and larger rerankers are preferable when higher compression ratios are needed.

Performance gains show diminishing returns as the compression ratio decreases. Reducing the compression ratio generally improves reranking performance because passage representations retain more information. However, the gains become smaller at lower compression ratios, indicating that these representations contain substantial redundancy. This suggests that moderate compression, such as $r = 2$ to $r = 8$, has little impact on reranking performance in most cases while substantially improving efficiency. These findings support using moderate compression ratios in practice instead of always keeping full representations, further confirming the effectiveness of the FBNL design.

Appendix D provides additional experiments on performance scaling with online computation cost.

6.4 In-depth Analysis

To further understand why reranking performance decreases as the compression ratio increases, we plot ROC curves over the reranked top-100 candidates on representative tasks in Figure 6. ROC-AUC evaluates whether the relevance scores assigned by a reranker can separate relevant passages from irrelevant ones, independent of a specific ranking cutoff. Clearly, as r increases, the AUC values consistently decrease, and the degradation is more pronounced for smaller models. For example, on FQA, when r increases from 2 to 32, the AUC of KaLM-Reranker-V1-Nano drops from 0.871 to 0.832, whereas KaLM-Reranker-V1-Large only decreases from 0.952 to 0.948. This indicates that under higher compression ratios, smaller models suffer a larger loss in distinguishing relevant passages from irrelevant ones. Focusing on the changes of r and AUC, the compression ratios can be roughly divided into two groups: the AUC values remain relatively stable for $r = 2, 4, 8$, but start to drop more clearly for $r = 16, 32$. This is consistent with the findings in §6.3: KaLM-Reranker-V1 maintains its discriminative ability under moderate compression, whereas overly aggressive compression causes noticeable information loss and performance degradation. This suggests exploring more effective compression methods that better preserve informative signals at high compression ratios.

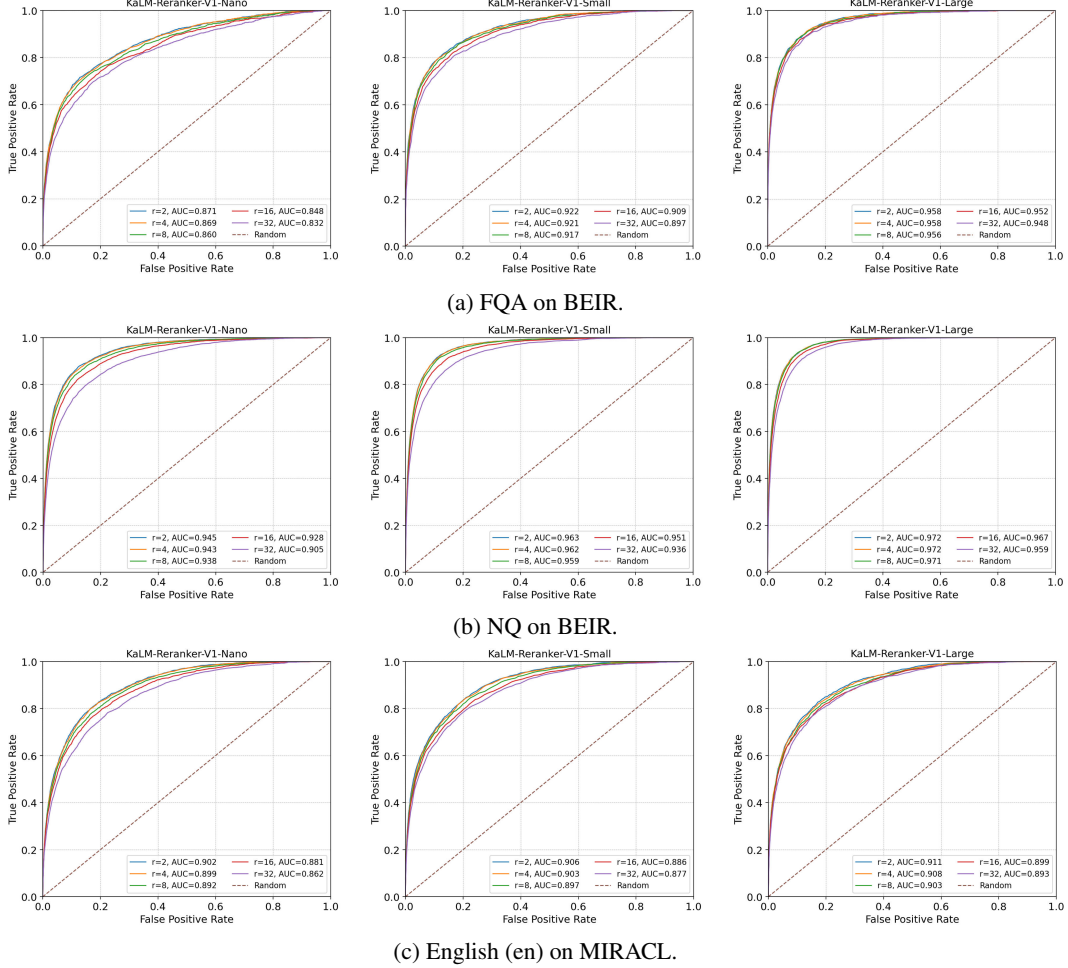


Figure 6: ROC curves on representative tasks from BEIR and MIRACL, where larger AUC values indicate a stronger ability to distinguish relevant passages from irrelevant ones.

7 Conclusion

We present the KaLM-Reranker-V1 series, a family of efficient rerankers built on the *fast but not late-interaction* (FBNL) paradigm. By decoupling passage encoding from decoder-side relevance modeling, KaLM-Reranker-V1 supports offline passage pre-encoding while preserving expressive relevance modeling through the decoder’s merged self-attention and cross-attention over the concatenation of the query context and pre-encoded passage representations. To further reduce storage and serving costs, we introduce Matryoshka embedding pooling (MEP), which compresses passage representations along the sequence dimension and enables flexible trade-offs between efficiency and effectiveness. Extensive experiments on BEIR, MIRACL, and LMEB show that the KaLM-Reranker-V1 series achieves competitive reranking performance compared with strong industrial rerankers while significantly reducing online overhead. Our in-depth analyses further show that moderate compression largely preserves discriminative ability, whereas overly aggressive compression can cause larger performance degradation, especially for smaller models. Future work includes developing more effective compression methods that preserve informative signals under high compression ratios, as well as further improving multilingual and Chinese reranking capabilities.

References

Mohammad Kalim Akram, Saba Sturua, Nastia Havriushenko, Quentin Herreros, Michael Günther, Maximilian Werk, and Han Xiao. jina-embeddings-v5-text: Task-targeted embedding distillation.

- arXiv preprint arXiv:2602.15547*, 2026.
- Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2016.
- Luiz Bonifacio, Vitor Jeronimo, Hugo Queiroz Abonizio, Israel Campiotti, Marzieh Fadaee, Roberto Lotufo, and Rodrigo Nogueira. mmarco: A multilingual version of the ms marco passage ranking dataset. *arXiv preprint arXiv:2108.13897*, 2021.
- Vera Boteva, Demian Gholipour, Artem Sokolov, and Stefan Riezler. A full-text learning to rank dataset for medical information retrieval. In *European Conference on Information Retrieval*, pages 716–722. Springer, 2016.
- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 4(5), 2024.
- Yiming Cui, Ting Liu, Wanxiang Che, Li Xiao, Zhipeng Chen, Wentao Ma, Shijin Wang, and Guoping Hu. A span-extraction dataset for chinese machine reading comprehension. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 5883–5889, 2019.
- Matthew Dunn, Levent Sagun, Mike Higgins, V Ugur Guney, Volkan Cirik, and Kyunghyun Cho. Searchqa: A new q&a dataset augmented with context from a search engine. *arXiv preprint arXiv:1704.05179*, 2017.
- Kenneth C. Enevoldsen, Isaac Chung, Imene Kerboua, Márton Kardos, Ashwin Mathur, David Stap, Jay Gala, Wissam Siblini, Dominik Krzeminski, Genta Indra Winata, Saba Sturua, Saiteja Utpala, Mathieu Ciancone, Marion Schaeffer, Diganta Misra, Shreeya Dhakal, Jonathan Rystrom, Roman Solomatin, Ömer Veysel Çagatan, Akash Kundu, and et al. MMTEB: massive multilingual text embedding benchmark. In *ICLR. OpenReview.net*, 2025.
- Felix Hamborg, Norman Meuschke, Corinna Breitingner, and Bela Gipp. *News-please*. Philosophische Fakultät, 2017.
- Faegheh Hasibi, Fedor Nikolaev, Chenyan Xiong, Krisztian Balog, Svein Erik Bratsberg, Alexander Kotov, and Jamie Callan. Dbpedia-entity v2: A test collection for entity search. In *Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval*, pages 1265–1268, 2017.
- Wei He, Kai Liu, Jing Liu, Yajuan Lyu, Shiqi Zhao, Xinyan Xiao, Yuan Liu, Yizhong Wang, Hua Wu, Qiaoqiao She, et al. Dureader: a chinese machine reading comprehension dataset from real-world applications. In *Proceedings of the workshop on machine reading for question answering*, pages 37–46, 2018.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Doris Hoogeveen, Karin M Verspoor, and Timothy Baldwin. Cqadupstack: A benchmark data set for community question-answering research. In *Proceedings of the 20th Australasian document computing symposium*, pages 1–8, 2015.
- Baotian Hu, Qingcai Chen, and Fangze Zhu. Lcsts: A large scale chinese short text summarization dataset. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 1967–1972, 2015.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022a.
- Xinshuo Hu, Zifei Shan, Xinpeng Zhao, Zetian Sun, Zhenyu Liu, Dongfang Li, Shaolin Ye, Xinyuan Wei, Qian Chen, Baotian Hu, and Min Zhang. KaLM-Embedding: Superior training data brings a stronger embedding model. *CoRR*, abs/2501.01028, 2025.

- Xuming Hu, Zhijiang Guo, GuanYu Wu, Aiwei Liu, Lijie Wen, and Philip S Yu. Chef: A pilot chinese dataset for evidence-based fact-checking. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3362–3376, 2022b.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022.
- Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. Pubmedqa: A dataset for biomedical research question answering. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 2567–2577, 2019.
- Jina AI. jina-reranker-v2-base-multilingual. <https://huggingface.co/jinaai/jina-reranker-v2-base-multilingual>, 2024.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 6769–6781, 2020.
- Daniel Khashabi, Amos Ng, Tushar Khot, Ashish Sabharwal, Hannaneh Hajishirzi, and Chris Callison-Burch. Gooaq: Open question answering with diverse answer types. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 421–433, 2021.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pages 39–48, 2020.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Nv-embed: Improved techniques for training llms as generalist embedding models. In *International Conference on Learning Representations*, volume 2025, pages 79310–79333, 2025.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474, 2020.
- Patrick Lewis, Yuxiang Wu, Linqing Liu, Pasquale Minervini, Heinrich Küttler, Aleksandra Piktus, Pontus Stenetorp, and Sebastian Riedel. Paq: 65 million probably-asked questions and what you can do with them. *Transactions of the Association for Computational Linguistics*, 9:1098–1115, 2021.
- Chaofan Li, Zheng Liu, Shitao Xiao, and Yingxia Shao. Making large language models a better foundation for dense retrieval. *arXiv e-prints*, pages arXiv–2312, 2023a.
- Xianming Li, Aamir Shakir, Rui Huang, Tsz-fung Andrew Lee, Julius Lipp, Benjamin Clavié, and Jing Li. Prorank: Prompt warmup via reinforcement learning for small language models reranking. *arXiv preprint arXiv:2506.03487*, 2025.

- Yudong Li, Yuqing Zhang, Zhe Zhao, Linlin Shen, Weijie Liu, Weiquan Mao, and Hui Zhang. Csl: A large-scale chinese scientific literature dataset. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 3917–3923, 2022.
- Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning. *CoRR*, abs/2308.03281, 2023b.
- Wing Lian, Bley Goodson, Eugene Pentland, Austin Cook, Chanvichet Vong, and “Teknium”. Openorca: An open dataset of gpt augmented flan reasoning traces. *Hugging Face dataset repository*, 2023.
- Hongcheng Liu, Yusheng Liao, Yutong Meng, and Yuhao Wang. Xiezhi: Chinese law large language model. https://github.com/LiuHC0428/LAW_GPT, 2023.
- Weiwen Liu, Yunjia Xi, Jiarui Qin, Fei Sun, Bo Chen, Weinan Zhang, Rui Zhang, and Ruiming Tang. Neural re-ranking in multi-stage recommender systems: A review. In *IJCAI*, pages 5512–5520. ijcai.org, 2022.
- Dingkun Long, Qiong Gao, Kuan Zou, Guangwei Xu, Pengjun Xie, Ruijie Guo, Jian Xu, Guanjuan Jiang, Luxi Xing, and Ping Yang. Multi-cpr: A multi domain chinese dataset for passage retrieval. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3046–3056, 2022.
- Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. Www’18 open challenge: financial opinion mining and question answering. In *Companion proceedings of the the web conference 2018*, pages 1941–1942, 2018.
- Chaitanya Malaviya, Subin Lee, Sihao Chen, Elizabeth Sieber, Mark Yatskar, and Dan Roth. Expertqa: Expert-curated questions and attributed answers. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3025–3045, 2024.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: massive text embedding benchmark. In *EACL*, pages 2006–2029. Association for Computational Linguistics, 2023.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- Jianmo Ni, Gustavo Hernández Ábrego, Noah Constant, Ji Ma, Keith Hall, Daniel Cer, and Yinfei Yang. Large dual encoders are generalizable retrievers. *arXiv preprint arXiv:2112.07899*, 2021.
- Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.
- OpenClaw Contributors. Your own personal ai assistant. any os. any platform. the lobster way. <https://github.com/openclaw/openclaw>, 2026. GitHub repository, accessed: 2026-06-17.
- Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.
- Ronak Pradeep, Sahel Sharifmoghaddam, and Jimmy Lin. Rankvicuna: Zero-shot listwise document reranking with open-source large language models. *arXiv preprint arXiv:2309.15088*, 2023.
- Yujia Qin, Zihan Cai, Dian Jin, Lan Yan, Shihao Liang, Kunlun Zhu, Yankai Lin, Xu Han, Ning Ding, Huadong Wang, et al. Webcpm: Interactive web search for chinese long-form question answering. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8968–8988, 2023.
- Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, et al. Large language models are effective text rankers with pairwise ranking prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 1504–1518, 2024.

- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 2383–2392, 2016.
- Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for squad. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, 2018.
- Jun Rao, Xv Meng, Liang Ding, Shuhan Qi, Xuebo Liu, Min Zhang, and Dacheng Tao. Parameter-efficient and student-friendly knowledge distillation. *IEEE Trans. Multim.*, 26:4230–4241, 2024.
- Chandan K Reddy, Lluís Màrquez, Fran Valero, Nikhil Rao, Hugo Zaragoza, Sambaran Bandyopadhyay, Arnab Biswas, Anlu Xing, and Karthik Subbian. Shopping queries dataset: A large-scale esca benchmark for improving product search. *arXiv preprint arXiv:2206.06588*, 2022.
- Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.
- Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. Colbertv2: Efficient and effective retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734, 2022.
- Chih Chieh Shao, Trois Liu, Yuting Lai, Yiying Tseng, and Sam Tsai. Drdcd: a chinese machine reading comprehension dataset. *arXiv preprint arXiv:1806.00920*, 2018.
- Zhihong Shao, Minlie Huang, Jiangtao Wen, Wenfei Xu, and Xiaoyan Zhu. Long and diverse text generation with planning-based hierarchical variational model. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3257–3268, 2019.
- Shivalika Singh, Freddie Vargus, Daniel D’souza, Börje F Karlsson, Abinaya Mahendiran, Wei-Yin Ko, Herumb Shandilya, Jay Patel, Deividas Mataciunas, Laura O’Mahony, et al. Aya dataset: An open-access collection for multilingual instruction tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11521–11567, 2024.
- Saba Sturua, Isabelle Mohr, Mohammad Kalim Akram, Michael Günther, Bo Wang, Markus Krimmel, Feng Wang, Georgios Mastrapas, Andreas Koukounas, Nan Wang, et al. jina-embeddings-v3: Multilingual embeddings with task lora. *arXiv preprint arXiv:2409.10173*, 2024.
- Hongxuan Tang, Hongyu Li, Jing Liu, Yu Hong, Hua Wu, and Haifeng Wang. Dureader_robust: A chinese dataset towards evaluating robustness and generalization of machine reading comprehension in real-world applications. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 955–963, 2021.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663*, 2021.
- James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. Fever: a large-scale dataset for fact extraction and verification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 809–819, 2018.
- Ustinian. Law question-answering dataset. <https://www.heywhale.com/mw/dataset/5e953ca8e7ec38002d02fca7>, 2020.
- Ellen Voorhees, Tasmeer Alam, Steven Bedrick, Dina Demner-Fushman, William R Hersh, Kyle Lo, Kirk Roberts, Ian Soboroff, and Lucy Lu Wang. Trec-covid: constructing a pandemic information retrieval test collection. In *ACM SIGIR Forum*, volume 54, pages 1–12. ACM New York, NY, USA, 2021.

- Henning Wachsmuth, Shahbaz Syed, and Benno Stein. Retrieval of the best counterargument without prior topic knowledge. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 241–251, 2018.
- David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, 2020.
- Feng Wang, Yuqing Li, and Han Xiao. Jina-reranker-v3: Last but not late interaction for listwise document reranking. *arXiv preprint arXiv:2509.25085*, 2025.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- Lucy Lu Wang, Kyle Lo, Yoganand Chandrasekhar, Russell Reas, Jiangjiang Yang, Doug Burdick, Darrin Eide, Kathryn Funk, Yannis Katsis, Rodney Michael Kinney, et al. Cord-19: The covid-19 open research dataset. In *Proceedings of the 1st Workshop on NLP for COVID-19 at ACL 2020*, 2020.
- Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*, pages 641–649, 2024.
- Xiaohui Xie, Qian Dong, Bingning Wang, Feiyang Lv, Ting Yao, Weinan Gan, Zhijing Wu, Xi-angsheng Li, Haitao Li, Yiqun Liu, et al. T2ranking: A large-scale chinese benchmark for passage ranking. In *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*, pages 2681–2690, 2023.
- Dongjie Yang, Ruifeng Yuan, Yuantao Fan, Yifei Yang, Zili Wang, Shusen Wang, and Hai Zhao. Refgpt: Dialogue generation of gpt, by gpt, and for gpt. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 2511–2535, 2023.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380, 2018.
- Biao Zhang, Paul Suganthan, Gaël Liu, Ilya Philippov, Sahil Dua, Ben Hora, Kat Black, Gus Martins, Omar Sanseviero, Shreya Pathak, Cassidy Hardin, Francesco Visin, Jiageng Zhang, Kathleen Kenealy, Qin Yin, Xiaodan Song, Olivier Lacombe, Armand Joulin, Tris Warkentin, and Adam Roberts. T5gemma 2: Seeing, reading, and understanding longer. *CoRR*, abs/2512.14856, 2025a.
- Meishan Zhang, Xin Zhang, Xinping Zhao, Shouzheng Huang, Baotian Hu, and Min Zhang. On the role of pretrained language models in general-purpose text embeddings: A survey. *arXiv preprint arXiv:2507.20783*, 2025b.
- Sheng Zhang, Xin Zhang, Hui Wang, Lixiang Guo, and Shanshan Liu. Multi-scale attentive interaction networks for chinese medical question answer selection. *IEEE access*, 6:74061–74071, 2018.
- Xin Zhang, Yanzhao Zhang, Dingkun Long, Wen Xie, Ziqi Dai, Jialong Tang, Huan Lin, Baosong Yang, Pengjun Xie, Fei Huang, et al. mgte: Generalized long-context text representation and reranking models for multilingual text retrieval. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1393–1412, 2024.
- Xin Zhang, Ziqi Dai, Mingxin Li, Yanzhao Zhang, Dingkun Long, Pengjun Xie, Meishan Zhang, Wenjie Li, and Min Zhang. Supervised fine-tuning or contrastive learning? towards better multi-modal LLM reranking. In *The Fourteenth International Conference on Learning Representations*, 2026a.
- Xinyu Zhang, Xueguang Ma, Peng Shi, and Jimmy Lin. Mr. tydi: A multi-lingual benchmark for dense retrieval. In *Proceedings of the 1st workshop on multilingual representation learning*, pages 127–137, 2021.

- Xinyu Zhang, Nandan Thakur, Odunayo Ogundepo, Ehsan Kamalloo, David Alfonso-Hermelo, Xiaoguang Li, Qun Liu, Mehdi Rezagholizadeh, and Jimmy Lin. Miracl: A multilingual retrieval dataset covering 18 diverse languages. *Transactions of the Association for Computational Linguistics*, 11:1114–1131, 2023.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025c.
- Ziying Zhang, Zihan Liao, Hang Yu, Peng Di, and Rui Wang. F2llm-v2: Inclusive, performant, and efficient embeddings for a multilingual world. *arXiv preprint arXiv:2603.19223*, 2026b.
- Xinping Zhao, Baotian Hu, Yan Zhong, Shouzheng Huang, Zihao Zheng, Meng Wang, Haofen Wang, and Min Zhang. Raserec: Retrieval-augmented sequential recommendation. *CoRR*, abs/2412.18378, 2024a.
- Xinping Zhao, Dongfang Li, Yan Zhong, Boren Hu, Yibin Chen, Baotian Hu, and Min Zhang. SEER: self-aligned evidence extraction for retrieval-augmented generation. In *EMNLP*, pages 3027–3041. Association for Computational Linguistics, 2024b.
- Xinping Zhao, Xinshuo Hu, Zifei Shan, Shouzheng Huang, Yao Zhou, Xin Zhang, Zetian Sun, zhenyu liu, Dongfang Li, Xinyuan Wei, Youcheng Pan, Yang Xiang, Meishan Zhang, Haofen Wang, Jun Yu, Baotian Hu, and Min Zhang. KaLM-embedding-v2: Superior training techniques and data inspire a versatile embedding model. In *The Fourteenth International Conference on Learning Representations*, 2026a.
- Xinping Zhao, Xinshuo Hu, Jiaxin Xu, Danyu Tang, Xin Zhang, Mengjia Zhou, Yan Zhong, Yao Zhou, Zifei Shan, Meishan Zhang, Baotian Hu, and Min Zhang. LMEB: long-horizon memory embedding benchmark. *CoRR*, abs/2603.12572, 2026b.
- Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhui Chen, and Xiang Yue. Opencodeinterpreter: Integrating code generation with execution and refinement. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12834–12859, 2024.
- Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. Lima: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36:55006–55021, 2023.

Hyperparameter	Value
Evaluation metric	nDCG@10
First-stage retriever	KaLM-Embedding-V2.5
Retrieved candidates per query K	100
Default compression ratio r	$r = 4$
Precision	bf16
Embedder query max length $ q $	512
Embedder passage max length n	512
Reranker query max length $ q $	512
Reranker passage max length n	1024

Table 6: Evaluation hyperparameter settings used in our experiments. Since bge-reranker-large only supports a maximum length of 512, its reranker query max length is set to 256, and its reranker passage max length is set to 512.

A Training Data

We fine-tune KaLM-Reranker-V1 on retrieval-specific datasets to develop its reranking capability. To improve robustness and generalization, we collect and process large-scale multilingual and multi-domain training data covering diverse retrieval scenarios, such as web search, question answering,

Model	Params.	#Layers	Hidden Dim.	Public Model Checkpoints (Link)
KaLM-Embedding-V2.5	0.5B	24	896	https://huggingface.co/KaLM-Embedding/KaLM-embedding-multilingual-mini-instruct-v2.5
bge-reranker-large	0.6B	24	1024	https://huggingface.co/BAAI/bge-reranker-large
bge-reranker-v2-m3	0.6B	24	1024	https://huggingface.co/BAAI/bge-reranker-v2-m3
bge-reranker-v2-gemma	2.5B	18	2048	https://huggingface.co/BAAI/bge-reranker-v2-gemma
gte-multilingual-reranker-base	0.3B	12	768	https://huggingface.co/Alibaba-NLP/gte-multilingual-reranker-base
Qwen3-Reranker-0.6B	0.6B	28	1024	https://huggingface.co/Qwen/Qwen3-Reranker-0.6B
Qwen3-Reranker-4B	4B	36	2560	https://huggingface.co/Qwen/Qwen3-Reranker-4B
Qwen3-Reranker-8B	8B	36	4096	https://huggingface.co/Qwen/Qwen3-Reranker-8B
jina-reranker-v2-base-multilingual	0.3B	12	768	https://huggingface.co/jinaai/jina-reranker-v2-base-multilingual
mxbai-reranker-base-v2	0.5B	24	896	https://huggingface.co/mixedbread-ai/mxbai-reranker-base-v2
mxbai-reranker-large-v2	1.5B	28	1536	https://huggingface.co/mixedbread-ai/mxbai-reranker-large-v2

Table 7: Model specifications and public checkpoints of the embedding and reranking models used in our evaluation.

duplicate-question retrieval, and fact verification. Our training data is mainly derived from two public sources: the retrieval subset of the KaLM embedding fine-tuning data⁶ and a selected subset of the BGE-M3 training data⁷. Since many retrieval datasets only provide query-positive pairs, we further mine hard negatives to construct reranking training instances. Specifically, for each query, we use KaLM-Embedding-V2.5 [Zhao et al., 2026a] to retrieve the top-100 candidate passages from the corresponding corpus, and sample 16 hard negatives from ranks 10–50. This sampling strategy encourages the model to distinguish relevant passages from hard negatives, while avoiding overly easy low-ranked negatives and reducing the risk of selecting false negatives from the highest-ranked candidates. After preprocessing, the final training data contains approximately 3.7M training samples. Detailed statistics of the training data are provided in Table 14.

B Implementation Details

The KaLM-Reranker-V1 series is initialized from the T5Gemma2 encoder-decoder backbone [Zhang et al., 2025a] and trained with LoRA [Hu et al., 2022a], where both the encoder and decoder parameters are fine-tuned. The LoRA target modules are `q_proj`, `k_proj`, `v_proj`, and `out_proj`. Specifically, KaLM-Reranker-V1-Nano, KaLM-Reranker-V1-Small, and KaLM-Reranker-V1-Large are built from the `t5gemma-2-270m-270m`, `t5gemma-2-1b-1b`, and `t5gemma-2-4b-4b` checkpoints, respectively. All models are trained with bf16 precision. We set the maximum query and passage lengths to 128 and 512 tokens, respectively. The training group size is set to 16, meaning that each training instance contains 1 positive passage and 15 negative passages. For Matryoshka Embedding Pooling (MEP), we train with compression ratios $\mathcal{R} = \{2, 4, 8, 16\}$ and use an equal loss weight $\lambda_r = 1$ for each ratio. We optimize the models with the Adam optimizer [Kingma and Ba, 2014] ($\beta_1 = 0.9$, $\beta_2 = 0.999$), using a weight decay of $1 \times e^{-5}$ and a cosine learning-rate scheduler.

We adopt the progressive multi-stage training pipeline described in §4.2. KaLM-Reranker-V1-Nano and KaLM-Reranker-V1-Small are trained through all three stages, whereas KaLM-Reranker-V1-Large is trained through only the first two stages. The learning rates for the first, second, and third stages are set to $1 \times e^{-4}$, $2 \times e^{-4}$, and $5 \times e^{-5}$, respectively, and each stage is trained for one epoch. In the distillation stage, KaLM-Reranker-V1-Large serves as the teacher model for training KaLM-Reranker-V1-Nano and KaLM-Reranker-V1-Small. The per-GPU batch sizes are 2, 2, and 1 for KaLM-Reranker-V1-Nano, KaLM-Reranker-V1-Small, and KaLM-Reranker-V1-Large, respectively, with 4 gradient accumulation steps. KaLM-Reranker-V1-Nano and KaLM-Reranker-V1-Small are trained on 16 NVIDIA RTX 5090 GPUs, while KaLM-Reranker-V1-Large is trained on 32 NVIDIA RTX 5090 GPUs, yielding an effective total batch size of 128 for each model. Training takes approximately 29k steps per epoch. The detailed training hyperparameters are summarized in Table 8.

C Instruction Templates

Tables 9, 10, and 11 summarize the instructions used for evaluation on BEIR, MIRACL, and LMEB, respectively.

⁶<https://huggingface.co/datasets/KaLM-Embedding/KaLM-embedding-finetuning-data>

⁷<https://huggingface.co/datasets/Shitao/bge-m3-data>

Hyperparameter	Nano	Small	Large
Training stages	3	3	2
Precision	bf16	bf16	bf16
Query max length $ q $	128	128	128
Passage max length n	512	512	512
MEP Compression ratios $\mathcal{R}$	{2, 4, 8, 16}	{2, 4, 8, 16}	{2, 4, 8, 16}
Loss weight λ_r for each $r \in \mathcal{R}$	1	1	1
Learning rate	Stage 1: $1 \times e^{-4}$	Stage 1: $1 \times e^{-4}$	Stage 1: $1 \times e^{-4}$
	Stage 2: $2 \times e^{-4}$	Stage 2: $2 \times e^{-4}$	Stage 2: $2 \times e^{-4}$
	Stage 3: $5 \times e^{-5}$	Stage 3: $5 \times e^{-5}$	
Batch size	2	2	1
Accumulation steps	4	4	4
Number of GPUs	16	16	32
GPU hours per epoch	~ 435	~ 780	~ 2250
LoRA rank	96	64	64
LoRA alpha	48	32	32
Number of epochs	1 epoch per stage	1 epoch per stage	1 epoch per stage
Warmup ratio	0.1	0.1	0.1
Backbone	t5gemma-2-270m-270m	t5gemma-2-1b-1b	t5gemma-2-4b-4b
Teacher model	KaLM-Reranker-V1-Large	KaLM-Reranker-V1-Large	–

Table 8: Training hyperparameters for the KaLM-Reranker-V1 series.

Task Name	Instruction Template
ArguAna (AA)	Given a claim, find documents that refute the claim.
Climate-FEVER (CF)	Given a claim about climate change, retrieve documents that support or refute the claim.
CQADupStack (CQA)	Given a question, retrieve detailed question descriptions from Stackexchange that are duplicates to the given question.
DBPedia (DB)	Given a query, retrieve relevant entity descriptions from DBPedia.
FEVER (FV)	Given a claim, retrieve documents that support or refute the claim.
FiQA2018 (FQA)	Given a financial question, retrieve user replies that best answer the question.
HotpotQA (HQA)	Given a multi-hop question, retrieve documents that can help answer the question.
MSMARCO (MS)	Given a query, retrieve documents that answer the query.
NFCorpus (NF)	Given a question, retrieve relevant documents that best answer the question.
NQ (NQ)	Given a question, retrieve Wikipedia passages that answer the question.
Quora (QR)	Given a question, retrieve questions that are semantically equivalent to the given question.
SCIDOCs (SD)	Given a scientific paper title, retrieve paper abstracts that are cited by the given paper.
SciFact (SF)	Given a scientific claim, retrieve documents that support or refute the claim.

Table 9: Task instructions used for evaluation on the BEIR [Thakur et al., 2021] benchmark.

D Performance Scaling with Cost

We further analyze how performance scales with online computation cost across model sizes and compression ratios. Figure 7 summarizes the trends on BEIR and MIRACL, from which we mainly have the following main findings:

Larger models do not provide a free lunch at similar cost. At comparable computational cost, switching to a larger model rarely brings performance gains. This suggests that increasing model capacity alone is not always cost-effective, as the gains must be balanced against the higher serving cost.

Performance exhibits diminishing returns as computation increases. As computation cost increases, reranking performance generally improves, but with diminishing marginal gains. Across both BEIR and MIRACL, $r = 4$ often provides a favorable trade-off, achieving performance close to $r = 2$ at substantially lower cost.

Smaller models are more sensitive to compression. Smaller models show larger performance changes as r varies. In particular, the Nano model fluctuates the most, suggesting that smaller models need more informative passage representations, while larger models better tolerate compression.

Task Name	Instruction Template
Arabic (ar), Bengali (bn), German (de), English (en), Spanish (es), Persian (fa), Finnish (fi), French (fr), Hindi (hi), Indonesian (id), Japanese (ja), Korean (ko), Russian (ru), Swahili (sw), Telugu (te), Thai (th), Yoruba (yo), Chinese (zh)	Given a query, retrieve documents that answer the query.

Table 10: Task instructions used for evaluation on the MIRACL [Zhang et al., 2023] benchmark.

Task Name	Instruction Template
ConvoMem, LoCoMo, LongMemEval, MemBench, REALTALK, TMD	We directly use the task instructions from LMEB [Zhao et al., 2026b].

Table 11: Task instructions used for evaluation on the LMEB [Zhao et al., 2026b] benchmark.

E Results on LMEB

Agentic memory is increasingly important for LLM-based agents, as systems such as Open-Claw [OpenClaw Contributors, 2026] rely on memory recall and utilization to provide personalized and customized services. As such, we further evaluate KaLM-Reranker-V1 on LMEB [Zhao et al., 2026b], a benchmark for long-horizon memory retrieval. From the results shown in Tables 12 and 13, we observe several interesting findings:

Reranking is important for memory retrieval. KaLM-Reranker-V1-Small improves the first-stage retriever by 12.35 nDCG@10 points on LMEB. On BEIR, the gain is 6.23 points. The larger gain on LMEB shows the value of reranking for long-horizon memory retrieval.

Memory retrieval remains challenging. Across the six dialogue memory retrieval tasks, no reranker achieves the best result on all tasks. Each model shows weaknesses on some datasets. This indicates that memory retrieval still needs further optimization.

Rerankers help more on complex queries. Specifically, TMD contains many temporal and time-based queries, including both relative and absolute time expressions. Such queries are difficult for embedding models. Rerankers handle them better. For example, Qwen3-Reranker-4B achieves 58.79 on TMD, while the first-stage retriever achieves only 16.82.

Retrieve-then-rerank is more effective than scaling embedding models. As shown in Table 13, the 9B bge-multilingual-gemma embedding model achieves an average score of 59.60. In contrast, the 0.5B KaLM-Embedding-V2.5 retriever plus the 0.27B KaLM-Reranker-V1-Nano reranker achieves 61.39. This demonstrates a clear advantage of the retrieve-then-rerank pipeline over simply scaling embedding models, indicating a synergistic effect between retrieval and reranking.

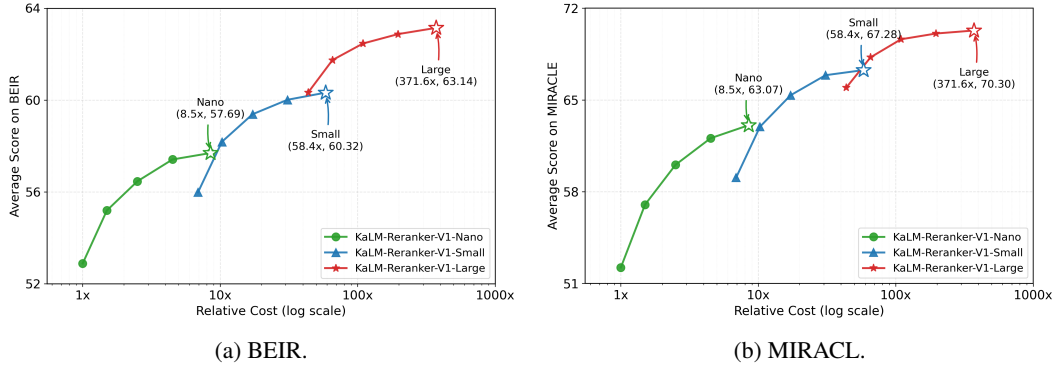


Figure 7: Performance scaling of the KaLM-Reranker-V1 series with online computation cost. Each curve traces how reranking performance scales as the cost increases across model sizes and Matryoshka compression pooling settings. The results are taken from Tables 15 and 16.

Models	Size	Cost	Avg.	ConvoMem	LoCoMo	LongMemEval	MemBench	REALTALK	TMD
First-stage Retriever									
KaLM-Embedding-V2.5	0.5B	—	50.80	62.74	41.88	75.18	69.59	38.60	16.82
Second-stage Reranker									
<i>Models with more than 4B parameters</i>									
Qwen3-Reranker-8B	8B	539.7x	66.42	64.64	65.91	78.93	76.31	57.13	55.58
<i>Models with 1B–4B parameters</i>									
Qwen3-Reranker-4B	4B	236.8x	62.86	66.68	63.43	53.72	78.65	55.91	58.79
KaLM-Reranker-V1-L	4B	43.7x	64.16	<u>68.83</u>	59.63	89.73	73.29	<u>55.15</u>	38.35
bge-reranker-v2-gemma	2.5B	81.3x	63.50	67.12	61.30	<u>87.23</u>	76.50	54.74	34.13
mxbai-rerank-large-v2	1.5B	79.2x	66.08	70.41	<u>61.33</u>	87.16	81.18	54.43	<u>41.96</u>
<i>Models with 0.5B–1B parameters</i>									
KaLM-Reranker-V1-S	1B	6.9x	<u>63.15</u>	65.81	59.86	87.58	74.73	<u>53.49</u>	<u>37.44</u>
bge-reranker-large	0.6B	36.3x	60.64	62.55	58.94	84.10	71.48	51.50	35.30
bge-reranker-v2-m3	0.6B	36.3x	62.38	<u>69.15</u>	<u>60.20</u>	82.39	<u>76.82</u>	53.90	31.84
Qwen3-Reranker-0.6B	0.6B	42.4x	63.69	67.36	60.32	84.91	75.67	53.26	40.63
mxbai-rerank-base-v2	0.5B	29.8x	62.47	69.41	55.76	<u>86.29</u>	79.97	49.75	33.61
<i>Models with fewer than 0.5B parameters</i>									
gte-reranker-base	0.3B	11.9x	56.08	<u>54.78</u>	51.01	78.94	68.76	47.97	<u>35.02</u>
jina-reranker-v2	0.3B	11.9x	<u>59.60</u>	54.70	<u>56.22</u>	86.55	<u>72.66</u>	<u>51.53</u>	35.97
KaLM-Reranker-V1-N	0.27B	1.0x	61.39	65.11	57.08	<u>85.52</u>	74.83	52.70	33.10

Table 12: Reranking results (nDCG@10) on the six dialogue memory retrieval tasks from LMEB [Zhao et al., 2026b]. “N”, “S”, and “L” denote Nano, Small, and Large, respectively. Avg. denotes the average performance across the six tasks. KaLM-Reranker-V1 adopts $r = 4$. Within each parameter group, the best results are **boldfaced**, and the second-best results are underlined.

Models	Size	Avg.	ConvoMem	LoCoMo	LongMemEval	MemBench	REALTALK	TMD
KaLM-Embedding-Gemma3 [Zhao et al., 2026a]	12B	56.59	64.08	42.77	81.44	73.79	44.38	33.10
bge-multilingual-gemma [Chen et al., 2024]	9B	59.60	67.11	56.11	81.69	72.95	48.64	31.10
Qwen3-Embedding-8B [Zhang et al., 2025c]	8B	51.69	55.43	36.35	77.59	71.31	41.22	28.22
NV-Embed-v2 [Lee et al., 2025]	7B	56.42	71.04	51.06	77.34	71.45	48.01	19.61
e5-mistral-7b-instruct [Wang et al., 2022]	7B	55.03	68.64	46.22	77.47	69.48	42.77	25.63

Table 13: Embedding retrieval results (nDCG@10) on the six dialogue memory retrieval tasks from LMEB [Zhao et al., 2026b]. Results are directly taken from the LMEB paper.

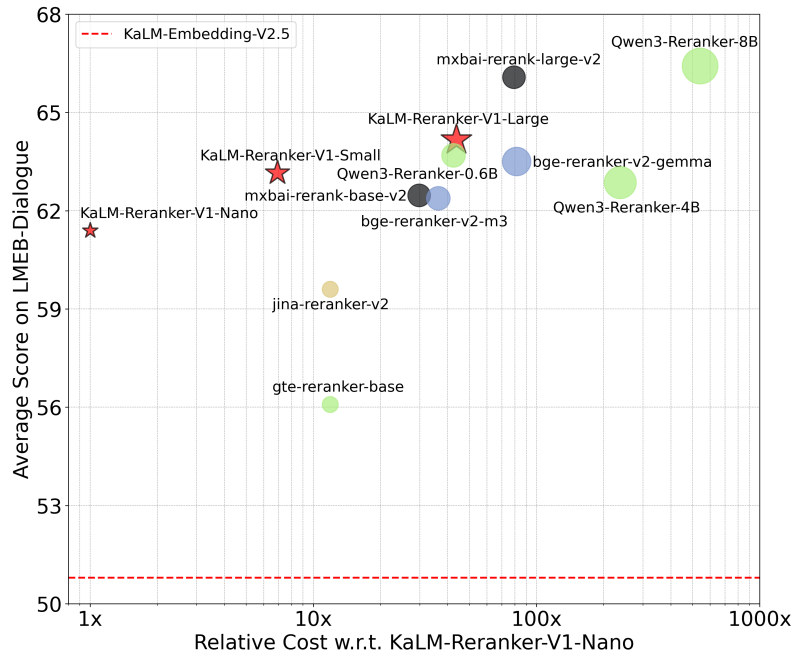


Figure 8: Comparison between the KaLM-Reranker-V1 series and other reranking models on LMEB-Dialogue [Zhao et al., 2026b] in terms of reranking performance and relative online computation cost. The cost is estimated following the analysis in §5.1; the x-axis is plotted on a logarithmic scale. Marker sizes are proportional to model sizes. The results are taken from Table 12.

Source	Language	Size	URL
KaLM embedding fine-tuning data (retrieval subset)			
AdvertiseGen [Shao et al., 2019]	zh	17,526	https://huggingface.co/datasets/shibing624/AdvertiseGen
CHEF [Hu et al., 2022b]	zh	4,824	https://github.com/THU-BPM/CHEF
CodeFeedback [Zheng et al., 2024]	en	49,090	https://huggingface.co/datasets/m-a-p/CodeFeedback-Filtered-Instruction
DRCD [Shao et al., 2018]	zh	4,714	https://huggingface.co/datasets/voidful/DRCD
Expertqa [Malaviya et al., 2024]	en	1,252	https://github.com/chaitanyamalaviya/ExpertQA
GooAQ [Khashabi et al., 2021]	en	49,833	https://github.com/allenai/gooaq
LCSTS [Hu et al., 2015]	zh	19,535	https://huggingface.co/datasets/hugcyp/LCSTS
MEDI2BGE [Xiao et al., 2024]	en	71,790	https://hf.co/datasets/GritLM/MEDI2BGE
Multi-CPR [Long et al., 2022]	zh	234,587	https://github.com/Alibaba-NLP/Multi-CPR
OpenOrca [Lian et al., 2023]	en	38,623	https://huggingface.co/datasets/Open-Orca/OpenOrca
PAQ [Lewis et al., 2021]	en	49,849	https://huggingface.co/datasets/sentence-transformers/paq
PubMedQA [Jin et al., 2019]	en	79,954	https://huggingface.co/datasets/qiaojin/PubMedQA
RefGPT [Yang et al., 2023]	zh	49,896	https://github.com/sufengniu/RefGPT
SearchQA [Dunn et al., 2017]	en	9,988	https://huggingface.co/datasets/kyunghyuncho/search_qa
T2Ranking [Xie et al., 2023]	zh	188,606	https://huggingface.co/datasets/THUIR/T2Ranking
THUCNews	zh	19,288	https://huggingface.co/datasets/SirlyDreameer/THUCNews
UMETrip-QA	zh	2,537	https://aistudio.baidu.com/datasetdetail/149933
WebCPM [Qin et al., 2023]	zh	1,602	https://github.com/thunlp/WebCPM
arxiv_qa	en	17,927	https://huggingface.co/datasets/TitanMLData/arxiv_qa
aya_dataset [Singh et al., 2024]	ml	26,292	https://huggingface.co/datasets/Coherelabs/aya_dataset
cCOVID-News	zh	4,727	https://www.datafountain.cn/competitions/424/datasets
cMedQA-V2.0 [Zhang et al., 2018]	zh	88,109	https://huggingface.co/datasets/wangrongsheng/cMedQA-V2.0
ccnews [Hamborg et al., 2017]	en	28,246	https://edoc.hu-berlin.de/items/ad915f2d-bb2c-4abd-887f-3d50bd3f2516
CMRC 2018 [Cui et al., 2019]	zh	9,753	https://huggingface.co/datasets/erhwenkuo/squad-cmrc2018-zhtw
cord19_trec-covid [Voorhees et al., 2021, Wang et al., 2020]	en	48,517	https://huggingface.co/datasets/irds/cord19_trec-covid
CQADupstack [Hoogeveen et al., 2015]	en	7,355	http://nlp.cis.unimelb.edu.au/resources/cquadupstack/
cs1 [Li et al., 2022]	zh	19,945	https://huggingface.co/datasets/neuclir/cs1
dbpedia-entity [Hasibi et al., 2017]	en	96,792	https://github.com/iai-group/DBpedia-Entity/
dureader [He et al., 2018]	zh	79,229	https://huggingface.co/datasets/sentence-transformers/dureader
dureader-checklist [Tang et al., 2021]	zh	97,764	https://huggingface.co/datasets/luozhouyang/dureader
esci [Reddy et al., 2022]	en	26,043	https://huggingface.co/datasets/tasksource/esci
fever [Thorne et al., 2018]	en	87,216	https://huggingface.co/datasets/maxzoech/fever
f1qa [Maia et al., 2018]	en	4,689	https://huggingface.co/datasets/irds/beir_f1qa_train
hotpot_qa [Yang et al., 2018]	en	150,153	https://huggingface.co/datasets/hotpotqa/hotpot_qa
law-gpt [Liu et al., 2023]	zh	500	https://huggingface.co/datasets/sentence-transformers/law-gpt
lawzhidao [Ustinian, 2020]	zh	6,784	https://www.heywhale.com/mw/dataset/5e953ca8e7ec38002d02fca7/content
lima-chinese [Zhou et al., 2023]	zh	1,991	https://huggingface.co/datasets/paralym/lima-chinese
miracl [Zhang et al., 2023]	ml	39,946	https://huggingface.co/datasets/sentence-transformers/miracl
mmarco-chinese [Bonifacio et al., 2021]	zh	379,870	https://huggingface.co/datasets/unicamp-dl/mmarco
mr-tydi [Zhang et al., 2021]	ml	46,997	https://huggingface.co/datasets/castorini/mr-tydi
msmarco-passage [Bajaj et al., 2016]	en	174,190	https://huggingface.co/datasets/Tevatron/msmarco-passage
msmarco-v2 [Bajaj et al., 2016]	en	258,617	https://huggingface.co/datasets/mteb/msmarco-v2
nfcopus [Boteva et al., 2016]	en	10,471	https://huggingface.co/datasets/BeIR/nfcopus-generated-queries
rag-dataset-12000	en	9,272	https://huggingface.co/datasets/neural-bridge/rag-dataset-12000
retrieval_data_llm_ingrad	zh	32,551	https://huggingface.co/datasets/ingrad/retrieval_data_llm
scifact [Wadden et al., 2020]	en	794	https://huggingface.co/datasets/Tevatron/scifact
squad_v2 [Rajpurkar et al., 2018, 2016]	en	125,816	https://huggingface.co/datasets/rajpurkar/squad_v2
triviaqa [Joshi et al., 2017]	en	44,442	https://huggingface.co/datasets/multi-train/emb-triviaqa-train
webgpt_comparisons [Nakano et al., 2021]	en	18,924	https://huggingface.co/datasets/openai/webgpt_comparisons
webqa	zh	4,988	https://huggingface.co/datasets/suolyer/webqa
wikipedia-nq [Kwiatkowski et al., 2019]	en	56,377	https://huggingface.co/datasets/Tevatron/wikipedia-nq
yahoo-answers	en	21,724	https://huggingface.co/datasets/sentence-transformers/yahoo-answers
quora-question-pairs	en	83,098	https://huggingface.co/datasets/AlekseyKorshuk/quora-question-pairs
arguana [Wachsmuth et al., 2018]	en	4,065	https://zenodo.org/records/3973258
BGE-M3-data			
MSMARCO [Bajaj et al., 2016]	en	476,968	https://huggingface.co/datasets/Shitao/bge-m3-data
NQ [Kwiatkowski et al., 2019]	en	58,554	https://huggingface.co/datasets/Shitao/bge-m3-data
HotpotQA [Yang et al., 2018]	en	84,228	https://huggingface.co/datasets/Shitao/bge-m3-data
Trivia [Joshi et al., 2017]	en	60,283	https://huggingface.co/datasets/Shitao/bge-m3-data

Table 14: Training data list, where “en”, “zh”, and “ml” denote English, Chinese, and multilingual data, respectively. For BGE-M3-data, we mainly use training samples with lengths in the range of 0–500.

Model	CR	Cost	Avg.	AA	CF	CQA	DB	FV	FQA	HQA	MS	NF	NQ	QR	SD	SF
KaLM-Reranker-V1-Nano	2	8.5x	57.69	64.41	36.06	45.91	46.43	91.47	48.38	81.74	43.84	38.09	66.19	89.74	20.55	77.21
	4	4.5x	57.41	64.22	36.20	45.62	46.10	91.34	48.14	81.29	43.30	37.56	65.70	89.52	20.55	76.84
	8	2.5x	56.46	62.32	36.12	44.76	45.19	90.97	46.38	80.40	42.56	36.98	64.56	89.14	19.62	75.04
	16	1.5x	55.19	61.04	35.09	43.44	42.42	90.28	45.55	78.58	40.97	35.89	61.71	88.67	19.17	74.70
	32	1.0x	52.88	59.84	32.91	41.46	39.44	86.74	42.84	74.71	37.81	35.56	56.58	88.37	18.51	72.74
KaLM-Reranker-V1-Small	2	58.4x	60.32	67.62	40.98	48.84	47.46	92.68	55.85	82.97	44.35	40.66	71.50	90.66	22.22	78.43
	4	30.9x	60.01	67.48	41.01	48.36	46.64	92.46	55.32	82.64	43.79	40.69	71.41	90.48	21.88	78.02
	8	17.2x	59.38	66.35	40.34	47.68	46.02	92.03	54.29	81.97	43.10	39.96	70.37	90.26	21.82	77.71
	16	10.3x	58.18	64.65	39.36	46.27	44.60	91.36	53.24	80.63	41.74	39.01	68.13	89.96	20.78	76.60
	32	6.9x	55.98	62.52	36.43	44.75	42.44	87.78	51.36	78.21	39.36	37.89	63.65	89.78	20.09	73.45
KaLM-Reranker-V1-Large	2	371.6x	63.14	75.15	39.39	53.46	51.41	92.81	62.96	84.31	46.37	42.04	75.84	91.14	25.08	80.89
	4	196.7x	62.87	74.39	38.56	53.05	50.87	92.86	62.63	84.10	46.13	41.99	75.72	91.00	24.88	81.12
	8	109.3x	62.46	73.52	38.35	52.57	50.18	92.99	61.67	83.74	45.27	41.97	74.90	90.77	24.82	81.26
	16	65.6x	61.74	71.62	38.58	51.63	49.08	92.77	61.67	82.98	44.53	41.51	73.28	90.57	24.26	80.19
	32	43.7x	60.33	70.57	37.61	50.31	47.17	91.56	60.09	81.41	42.34	39.90	70.61	90.41	23.31	79.00

Table 15: BEIR [Thakur et al., 2021] reranking results under different Matryoshka embedding pooling compression ratios r , measured by nDCG@10. The 13 task abbreviations follow Table 9. “CR” denotes the compression ratio, where a larger value indicates more compressed passage representations. Cost denotes the estimated relative online computation cost derived from the time complexity analysis in §5.1, using $|q| = 32$, $n = 1024$, $K = 1$, and the corresponding compression ratio r , with L and d obtained from Table 1, and normalized to Nano at $r = 32$ as 1.0x. Avg. denotes the average performance across the 13 tasks.

Model	CR	Cost	Avg.	ar	bn	de	en	es	fa	fi	fr	hi	id	ja	ko	ru	sw	te	th	yo	zh
KaLM-Reranker-V1-N	2	8.5x	63.07	69.35	72.00	53.48	61.25	58.01	52.33	73.26	54.03	54.90	52.46	63.79	64.61	62.20	65.06	74.81	71.26	80.19	52.21
	4	4.5x	62.08	69.00	70.91	50.45	60.41	56.93	52.53	73.11	51.93	53.87	51.58	62.81	65.02	61.30	64.63	73.67	70.16	78.22	50.90
	8	2.5x	60.05	67.75	69.27	48.89	58.58	54.57	50.81	70.25	50.11	53.05	50.19	60.67	60.91	59.76	60.17	71.69	68.57	75.72	49.90
	16	1.5x	57.01	65.30	65.88	45.57	56.28	51.80	48.52	65.82	49.17	49.92	47.58	58.19	58.78	56.22	54.03	69.34	64.72	74.08	44.94
	32	1.0x	52.21	60.10	62.36	40.54	51.74	48.48	43.64	58.35	46.01	47.16	44.24	51.93	52.12	51.40	48.31	65.31	58.99	68.28	40.71
KaLM-Reranker-V1-S	2	58.4x	67.28	74.65	77.07	59.10	62.73	58.74	56.26	78.16	57.40	59.75	53.36	69.03	65.52	71.21	75.27	80.40	76.92	84.60	50.93
	4	30.9x	66.89	73.49	77.06	58.10	62.09	58.83	56.43	77.81	57.21	59.54	53.08	68.84	65.46	69.93	74.63	80.00	76.94	83.87	50.73
	8	17.2x	65.36	72.19	75.18	56.81	61.16	56.70	54.18	77.36	54.67	58.12	52.41	66.92	63.46	68.51	73.20	78.70	76.74	79.24	50.84
	16	10.3x	62.96	70.28	72.70	54.01	58.78	54.23	52.49	75.77	51.03	55.08	49.79	65.03	61.04	65.87	71.54	76.57	74.42	75.95	48.69
	32	6.9x	59.07	66.18	67.92	51.14	55.14	50.96	50.09	71.41	48.07	51.39	47.11	59.66	56.95	61.22	66.72	70.95	69.66	72.69	45.96
KaLM-Reranker-V1-L	2	371.6x	70.30	77.68	80.66	64.61	64.08	61.09	60.29	79.79	58.12	64.32	54.94	71.66	70.59	73.58	77.95	83.19	81.23	90.36	51.26
	4	196.7x	70.07	77.69	79.98	63.60	63.54	60.51	60.62	80.00	58.49	64.01	54.76	72.04	69.86	73.57	78.25	82.55	81.57	88.66	51.54
	8	109.3x	69.63	77.34	79.97	62.27	62.57	59.36	60.27	79.74	57.30	62.95	53.87	72.18	69.15	73.06	77.82	82.72	82.00	87.57	53.17
	16	65.6x	68.25	76.83	78.08	59.84	61.32	57.97	58.43	78.46	56.00	60.24	52.39	71.88	66.85	71.47	76.00	82.24	81.22	86.22	53.08
	32	43.7x	65.94	74.03	74.73	57.13	58.70	56.16	57.63	75.99	54.32	59.35	49.93	67.21	64.56	68.63	74.80	79.08	79.62	83.01	52.08

Table 16: MIRACL [Zhang et al., 2023] reranking results under different Matryoshka embedding pooling compression ratios r , measured by nDCG@10. All 18 language abbreviations follow Table 10. “CR” denotes the compression ratio, where a larger value indicates more compressed passage representations. Cost denotes the estimated relative online computation cost derived from the time complexity analysis in §5.1, using $|q| = 32$, $n = 1024$, $K = 1$, and the corresponding compression ratio r , with L and d obtained from Table 1, and normalized to Nano at $r = 32$ as 1.0x. Avg. denotes the average performance across the 18 languages.